

4. Creating another entity and biding both togeather.

fourth in series.

=====

I have created another entity called item_details.

I have created a table in mysql called item_details... id column of item table is foreign key in item_details table.

Item_detail table (id(primary key), item_id(foreign_key), category, available_colors).

```
CREATE TABLE item_detail (  
id INT PRIMARY KEY auto_increment,  
item_id INT,  
category varchar2(255),  
available_colors varchar2(255),  
FOREIGN KEY (item_id) REFERENCES item(id)  
);
```

=====

Next I created ItemDetail model with all the fields.

Also mentioned its relation with Item table.

```
@OneToOne  
@JoinColumn(name = "item_id", referencedColumnName = "id")  
@JsonBackReference  
private Item item;
```

@OneToOne says that relation between Item and Item_detail is one to one.

@JoinColumn tells that Item_detail has item_id column which is foreign key for **Item.id** (item table's id column).

You want to tell JPA:

"Hey, in the `ItemDetail` table, the column `item_id` is a foreign key pointing to `Item.id`."

That's exactly what `@JoinColumn` does.

Detailed Breakdown

java

Copy

Edit

```
@JoinColumn(  
    name = "item_id",           // column in THIS table (ItemDetail)  
    referencedColumnName = "id" // column in TARGET table (Item)  
)
```

Part

Meaning



`name = "item_id"`

This is the **foreign key column** in the current table (`ItemDetail`).

`referencedColumnName = "id"`

This is the **primary key column** in the target table (`Item`). By default it's `id` , so this is often optional.

@JoinColumn appears on owning table... means where foreign key is defined...

*****DO NOT FORGET TO DECLARE GETTER SETTERS, OTHERWISE
JSON WILL COME BLANK*****

Next, I created ItemDetailRepository... nothing fancy.. same as
ItemRepository.

***@JsonBackReference. — did not go in detail... its something to avoid
recursive call... even its ok to remove it for my case.***

=====

Now we want item_details to be fetched as well when we make call to Item.....

So I have created field

```
@OneToOne(mappedBy = "item", cascade = CascadeType.ALL,
fetch = FetchType.EAGER)
@JsonManagedReference
private ItemDetail itemDetail;
```

```
@OneToOne(mappedBy = "item", cascade = CascadeType.ALL, fetch =
FetchType.EAGER)
```

You're telling JPA:

"Each Item has one ItemDetail, and this relationship is managed (owned) by the item field inside the ItemDetail entity." .. note that we are talking about item field in ItemDetail... we have created a field Item there...i.e.

@JoinColumn one..

So, mappedBy = "item" tells JPA to **not create a new foreign key column** in the Item table. Instead, JPA will use the relationship defined in ItemDetail.

 **What Does `CascadeType.ALL` Do?**

java

Copy Edit

`CascadeType.ALL = { PERSIST, MERGE, REMOVE, REFRESH, DETACH }`

So using `CascadeType.ALL` means:

Cascade Type	What It Does
<code>PERSIST</code>	When you save the parent (<code>itemRepo.save(item)</code>), it also saves the child (<code>ItemDetail</code>) automatically
<code>MERGE</code>	When you update the parent, updates are also applied to the child
<code>REMOVE</code>	When you delete the parent, the child is deleted too
<code>REFRESH</code>	If you refresh the parent from the DB, the child is refreshed too
<code>DETACH</code>	If you detach the parent from the persistence context, the child is detached too

3. `fetch = FetchType.EAGER`

This controls when the associated `ItemDetail` should be loaded.

Type	Behavior
<code>EAGER</code>	Always load <code>ItemDetail</code> whenever <code>Item</code> is fetched
<code>LAZY</code>	Load <code>ItemDetail</code> only when you explicitly access it

With `EAGER`, JPA will do something like:

```
sql
SELECT * FROM item
LEFT JOIN item_detail ON item.id = item_detail.item_id
```

This can be good for tightly-coupled objects, but dangerous for performance if overused.

I did not go in detail for this :

4. `@JsonManagedReference`

This comes from Jackson and is used to prevent **infinite recursion** in JSON serialization when you have **bidirectional relationships**.

Think of it as the "parent" side.

You paired it with `@JsonBackReference` in `ItemDetail`:

```
java
@OneToOne
@JoinColumn(name = "item_id")
@JsonBackReference
private Item item;
```

This tells Jackson:

- Serialize `itemDetail` from `Item`
- But don't go back from `ItemDetail → Item → ItemDetail → ...` infinitely

Without these, you'd get a `StackOverflowError` during JSON serialization.



After doing this much, get call to item ..i.e. `findAll()` will automatically start

fetching item_details as well... !!!!!

🧠 Summary Table		
Annotation	Meaning	Typical DB Representation
<code>@OneToOne</code>	One entity ↔ One entity	Foreign key on one side
<code>@OneToMany</code>	One entity → Many entities	Foreign key on "many" side
<code>@ManyToOne</code>	Many entities → One entity	Foreign key on "many" side
<code>@ManyToMany</code>	Many ↔ Many	Join table (e.g., <code>student_course</code>)

One person will have one resident address — OnetoOne

One author can write many books — OnetoMany

Many books can be written by one auther — ManytoOne

ManytoMany — Each **Student** can enroll in multiple **Courses**, and each **Course** can have multiple **Students**.

📦 Summary		📄
Constraint Type	Examples	
Column Constraints	<code>@Column(nullable, unique, length)</code>	
Validation Constraints	<code>@NotNull</code> , <code>@Size</code> , <code>@Email</code> , <code>@Pattern</code>	
Relationship Constraints	<code>@JoinColumn</code> , <code>@ManyToOne</code>	
Custom Constraints	Your own <code>@ValidXYZ</code> validator	
Table-level Constraints	<code>@Table(uniqueConstraints = ...)</code>	